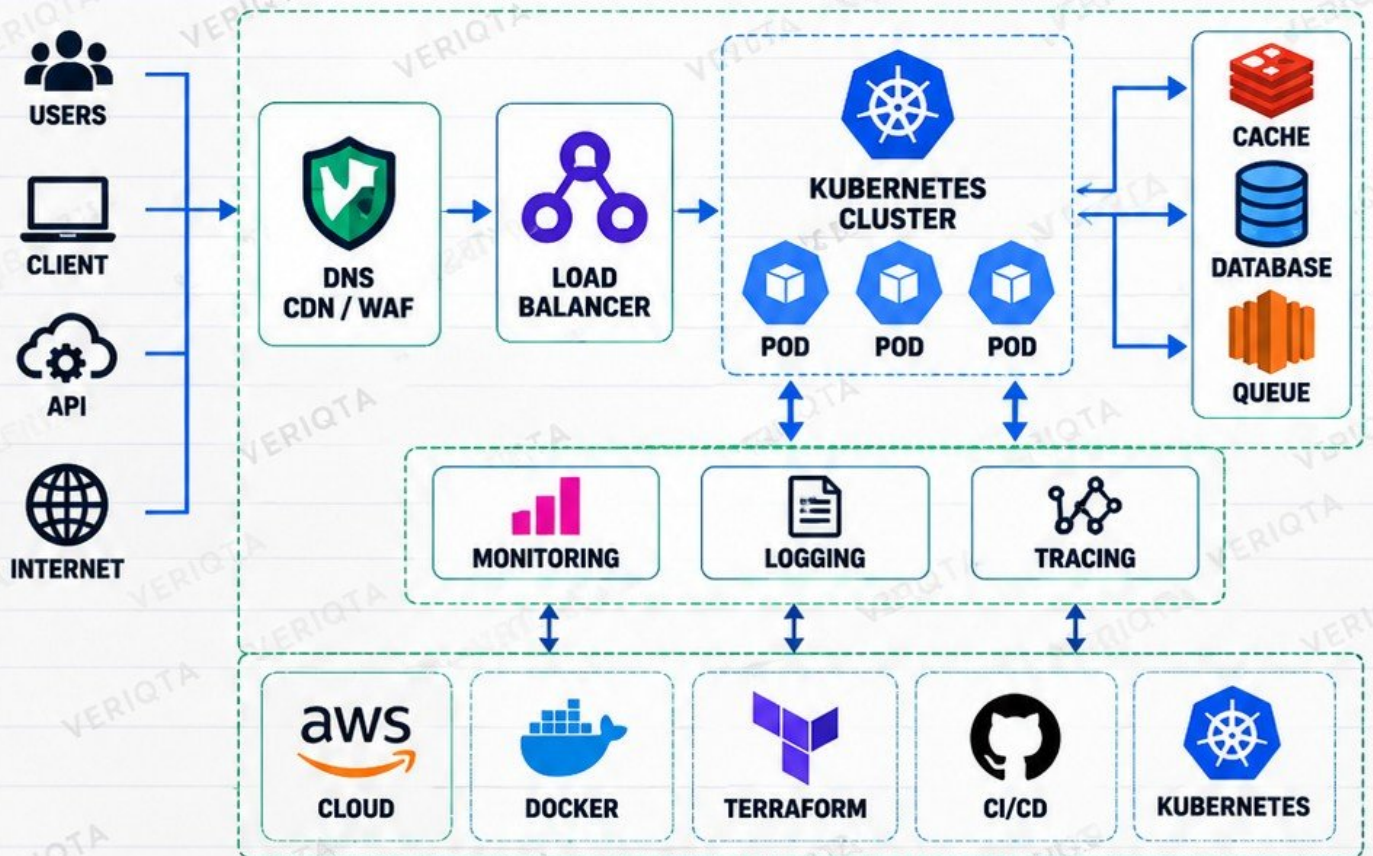


PRODUCTION ARCHITECTURE FOR DEVOPS ENGINEERS



**REAL-WORLD
ARCHITECTURE DESIGNS**



**RELIABILITY,
SCALABILITY & SECURITY**



**CLOUD & KUBERNETES
ARCHITECTURES**



**OBSERVABILITY
& OPERATIONS**



**RESILIENCE
& DISASTER RECOVERY**



**DEPLOYMENT
& RELEASE STRATEGIES**

**20
PAGES**

01

HOW SENIOR ENGINEERS READ ARCHITECTURE

1 START WITH THE REQUIREMENTS



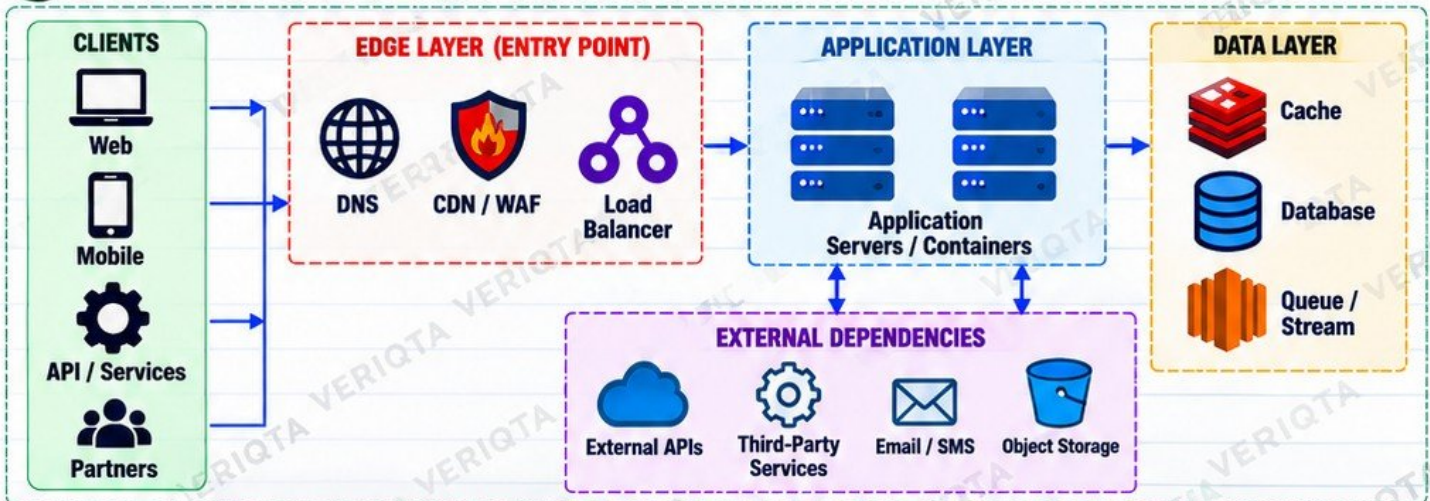
- What problem are we solving?
- Expected traffic and growth?
- Availability target (e.g. 99.9%)
- Performance and latency requirements?
- Data volume and retention?
- Compliance or regulatory needs?
- Cost constraints?
- Team ownership and operational model?

2 IDENTIFY CONSTRAINTS



- Budget limits
- Required technology stack
- Existing systems to integrate with
- Organizational policies
- Regulatory requirements
- Time to deliver
- Team skills and tooling
- Vendor or cloud limitations
- Data residency or regional constraints

3 LOOK AT THE BIG PICTURE



4 MAP DEPENDENCIES



- What depends on what?
- Internal and external dependencies
- Synchronous vs asynchronous
- Critical vs non-critical dependencies
- How failures propagate
- Which dependencies can be isolated?

5 DEFINE TRUST BOUNDARIES



- Public vs private networks
- User to system trust boundaries
- Internal service boundaries
- Data classification and sensitivity
- Access control requirements
- Cross-account or cross-environment access
- Third-party trust boundaries

6 IDENTIFY DATA PATHS



- Where does data enter the system?
- How does it flow through the system?
- Where is it stored?
- How is it processed and transformed?
- Where does it leave the system?
- What are the security and compliance requirements at each stage?

7 ANALYZE FAILURE DOMAINS



- Which components can fail independently?
- What is the blast radius of each failure?
- Single points of failure?
- Regional or availability zone dependencies?
- How do we isolate failures?
- What is the recovery strategy?

8 FIND THE CRITICAL PATHS



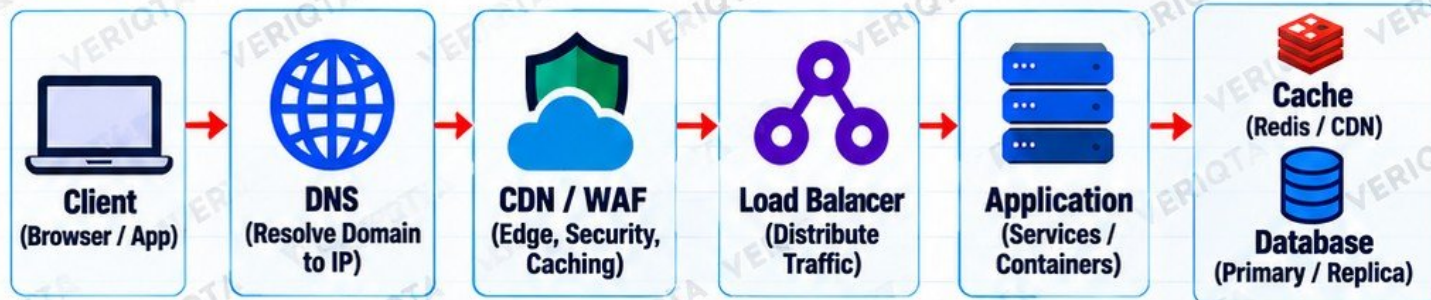
- What components are required for the core user journey?
- Which dependencies must be highly available?
- Where will latency have the biggest impact?
- What needs the strongest monitoring and redundancy?



SENIOR ENGINEER TIP

A good architecture is not about using the most services. It is about meeting the requirements, controlling the risks, and keeping the system simple enough to operate, scale and recover.

02 From Request to Response



1 DNS: Resolve Domain to IP

- Client queries DNS for the domain.
- Returns IP address (A/AAAA record).
- Result is cached based on TTL.
- Can use recursive and authoritative DNS.

Common Failures

- DNS timeout or no response
- Incorrect DNS record
- Expired or low TTL causing stale IP
- DNS propagation delays

2 CDN / WAF: Edge and Security

- CDN serves cached content from edge locations.
- WAF inspects and blocks malicious traffic.
- Reduces latency and protects origin.
- Can terminate TLS.

Common Failures

- Edge node unavailable
- WAF blocking legitimate traffic
- TLS certificate issues
- Cache misconfiguration (stale content)

3 Load Balancer: Distribute Traffic

- Forwards requests to healthy targets.
- Performs health checks.
- Can be Layer 4 (TCP) or Layer 7 (HTTP).
- Distributes traffic across AZs and nodes.

Common Failures

- Unhealthy targets
- Misconfigured health checks
- Cross-AZ issues
- Connection draining not configured

4 Application: Process Request

- Request is handled by application servers or containers.
- May call internal or external services.
- Reads from cache or database.
- Returns response to the client.

Common Failures

- Application errors (5xx)
- High latency due to dependencies
- Resource exhaustion (CPU, memory)
- Misconfigured environment or rollout

5 Cache: Improve Performance

- Stores frequently used data in memory (e.g. Redis, CDN).
- Reduces load on database and improves latency.
- Uses TTL for expiry.
- Can be cache-aside, write-through, etc.

Common Failures

- Cache miss or low hit rate
- Expired or stale data
- Cache node failure
- Increased latency to database

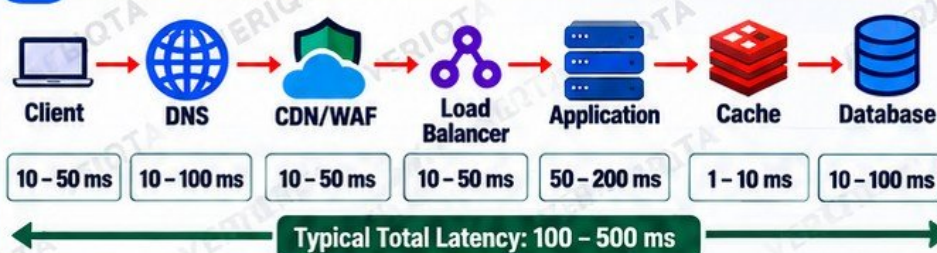
6 Database: Persistent Storage

- Stores and retrieves application data.
- Can use primary and read replicas.
- Requires connection pooling.
- Should be in private subnets.

Common Failures

- Slow queries
- Connection limit reached
- Primary or replica failure
- Storage full or high I/O latency

End-to-End Request Flow with Latency



Where Latency Increases

- Slow DNS resolution
- CDN cache miss
- WAF inspection overhead
- Load balancer queueing
- Application processing time
- Cache miss (goes to database)
- Slow database query
- Network latency between services
- Cross-region or cross-AZ traffic

Production Engineering Tips

- Use DNS health checks and appropriate TTLs.
- Enable CDN and cache static content.
- Use a WAF with managed rules.
- Deploy load balancers across multiple AZs.
- Keep applications stateless and scalable.
- Use caching to reduce database load.
- Monitor each hop: latency, error rate, saturation.
- Set timeouts and retries for external dependencies.

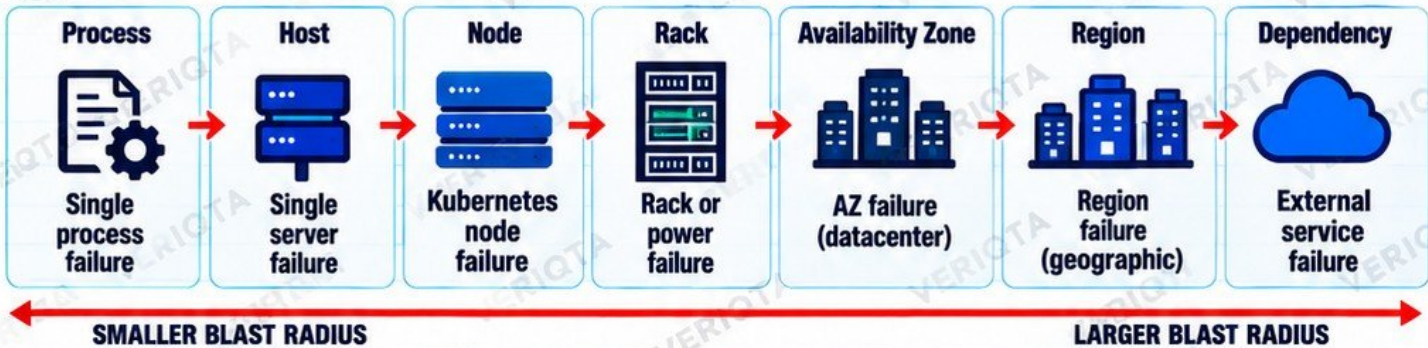
Key Metrics to Monitor

- DNS resolution time and failure rate
- CDN cache hit ratio and edge latency
- WAF blocked requests
- Load balancer latency and 5xx errors
- Application response time and error rate
- Cache hit ratio and latency
- Database query latency, connection count, CPU and I/O

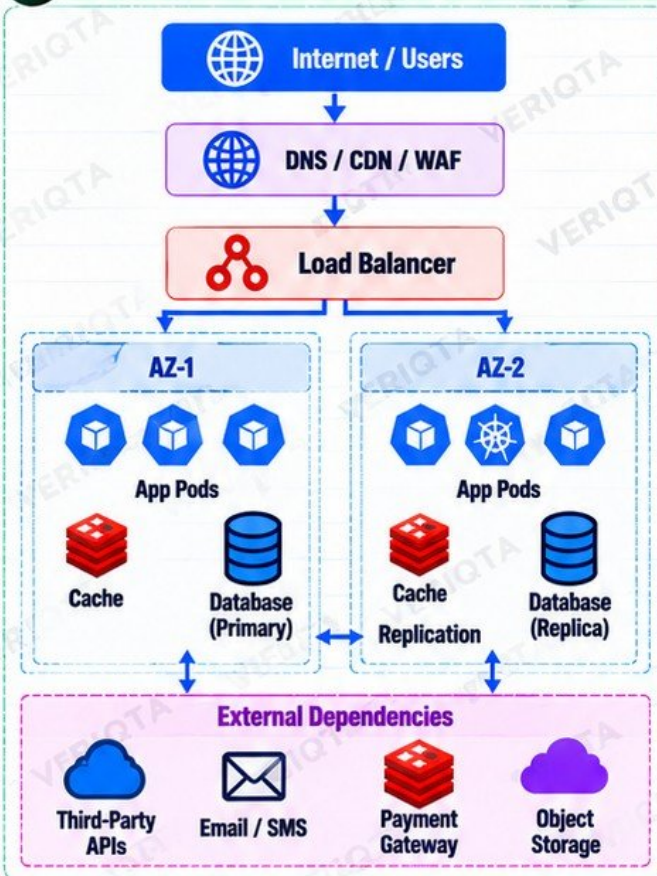
03 Designing for Failure Domains

Process, host, node, rack, AZ, region and dependency failures, limiting blast radius

1 FAILURE DOMAIN HIERARCHY



2 EXAMPLE ARCHITECTURE WITH FAILURE DOMAINS



3 COMMON FAILURES BY DOMAIN

Domain	Examples of Failures	Impact	Typical Blast Radius
Process	Crash, memory leak, high CPU	Service unavailable	Single process
Host	OS crash, hardware failure, disk failure	Multiple services down	Single server
Node	Kubernetes node failure, network loss	All pods on node affected	Single node
Rack	Power, network or switch failure	Multiple nodes down	Single rack
Availability Zone	Datacenter outage, network partition	Applications in AZ affected	Single AZ
Region	Natural disaster, regional outage	Full region affected	Entire region
Dependency	Database, external API, DNS, cloud service	Dependent services fail	Application or multi-region

4 LIMITING BLAST RADIUS

GOOD PRACTICES	ANTI-PATTERNS
- Use multiple AZs and regions where required - Keep services stateless where possible - Isolate workloads and dependencies - Use timeouts, retries and circuit breakers - Implement bulkheads and rate limiting - Design for graceful degradation - Use clear ownership boundaries - Monitor each layer independently	- All components in a single AZ - Shared databases across many unrelated services - No dependency isolation - Unlimited retries causing cascading failures - Tight coupling between services - No fallback strategy - Single region deployment - Ignoring external dependency risks

5 DESIGN QUESTIONS

- What is the failure domain for each component?
- How does a failure propagate?
- What is the expected blast radius?
- Which dependencies are critical?
- Can the system continue with partial failure?
- What needs multi-AZ or multi-region design?
- What monitoring will detect failures at each layer?
- How will we recover from each type of failure?

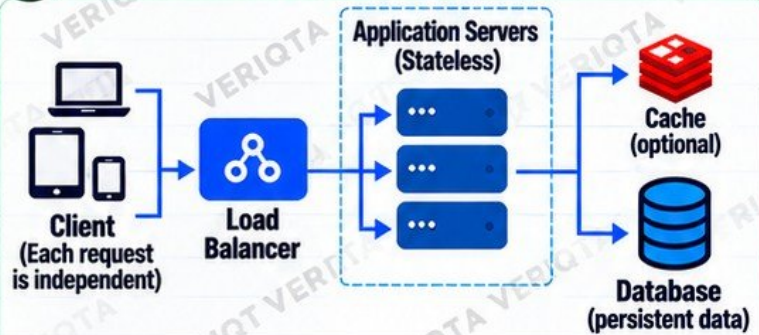
6 SENIOR ENGINEER TAKEAWAYS

- You cannot prevent all failures, but you can contain them.
- Design for the most likely failure first.
- Keep failure domains small and independent.
- Use redundancy, isolation and decoupling.
- Understand dependency failure behavior (timeouts, retries, limits).
- Plan for regional outages if the service is critical.
- Test failure scenarios regularly.
- A good architecture keeps the system running and limits user impact.

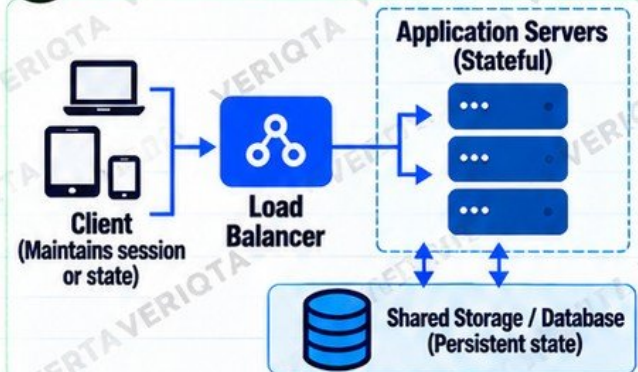
04 Stateless vs Stateful Architecture

State placement, persistence, horizontal scaling, session management, recovery and operational consequences

1 STATELESS ARCHITECTURE



2 STATEFUL ARCHITECTURE



3 KEY DIFFERENCES

ASPECT	STATELESS	STATEFUL
State location	No state on server	State stored on server or external system
Persistence	External (database, cache)	Local or external storage
Request handling	Each request independent	Requests depend on state
Horizontal scaling	Easy, add more servers	More complex, requires state sharing or replication
Session management	Use tokens (e.g. JWT)	Server-side sessions or shared store (e.g. Redis)
Failure recovery	Simpler, replace instance	Complex, must preserve or replicate state
Examples	Web APIs, microservices, static content	Databases, queues, stateful applications, file systems
Operational impact	Lower complexity	Higher complexity, more management

4 WHEN TO USE EACH

✓ USE STATELESS WHEN - You need high scalability. - Each request can be processed independently. - You can store state externally (e.g. database, cache). - You want simpler deployments and recovery. - You are building APIs, microservices or web apps. - You need to handle unpredictable traffic.	✗ USE STATEFUL WHEN - The application requires session or user state. - State cannot be easily externalized. - You need low-latency access to local state. - You are building databases, queues, or file systems. - You need strong consistency or transactional state. - You can manage replication, backup and failover.
---	---

5 COMMON STATE PLACEMENT PATTERNS

IN-MEMORY CACHE - Sessions - Temporary data - Fast access - Use for stateless services	EXTERNAL DATABASE - User data - Application data - Durable storage - Shared across instances	DISTRIBUTED STORAGE - Files and media - Shared access - Scalable - Use object storage (e.g. S3, GCS)	QUEUE / STREAM - Background tasks - Event processing - Decoupling - Reliable and durable
---	---	---	---

6 CHALLENGES WITH STATEFUL SYSTEMS

! - Harder to scale horizontally - Data consistency and replication - Longer recovery time - Split-brain and leader election issues - Storage performance and I/O limits - Backup and restore complexity - Handling failover without data loss - Increased operational overhead
--

7 DESIGN CONSIDERATIONS

⚙️ - What state needs to be stored and why? - Can the state be externalized? - How will you handle scaling and failover? - What is the expected load and growth? - What are the consistency and durability requirements? - How will you back up and recover the state? - What are the latency and performance requirements? - What are the security and compliance requirements?

8 SENIOR ENGINEER TAKEAWAYS

💡 - Prefer stateless design for scalability and simplicity. - Externalize state for better resilience. - Use stateful design only when required. - Design for failure: assume instances, nodes or AZs will fail. - Implement replication, backups and tested recovery for stateful systems. - Understand the trade-offs: performance vs consistency vs complexity. - Keep state small and isolated. - Document recovery procedures and test them regularly.
--

05 Load Balancing & Traffic Architecture

L4 vs L7, health checks, connection draining, algorithms, sticky sessions, failure behavior

1 WHAT IS A LOAD BALANCER

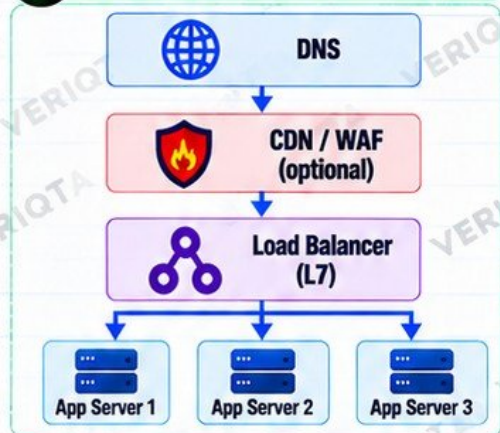
- Distributes incoming traffic across multiple servers.
- Improves availability, performance and scalability.
- Can operate at Layer 4 (Transport) or Layer 7 (Application).
- Performs health checks and removes unhealthy targets.
- Helps with connection management, SSL termination, and routing.



2 L4 vs L7 LOAD BALANCING

Aspect	Layer 4 (L4)	Layer 7 (L7)
Operates at	Transport layer (TCP/UDP)	Application layer (HTTP/HTTPS)
Routes based on	IP address, port	URL, host header, path, cookies, HTTP headers
Performance	Faster, lower latency	Slightly higher latency
Use cases	High performance, simple routing	Content-based routing, complex rules, web applications
Examples	NLB, HAProxy (TCP), AWS NLB	ALB, HAProxy (HTTP), NGINX, Traefik

3 TRAFFIC FLOW (L7 EXAMPLE)



4 LOAD BALANCING ALGORITHMS

Round Robin

- Distributes evenly
- Simple and common

Least Connections

- Sends to server with fewest active connections
- Good for variable traffic

Weighted Round Robin

- Distributes based on weights
- Useful when servers have different capacity

IP Hash

- Same client IP goes to same server
- Used for sticky sessions

Path / Header Based (L7)

- Routes based on URL, headers, cookies
- Used for microservices

5 HEALTH CHECKS

- Load balancer periodically checks target health.
- Can use TCP, HTTP or HTTPS checks.
- Unhealthy targets are removed from rotation.
- Helps prevent traffic from going to failed servers.
- Configure appropriate timeouts and thresholds.

6 CONNECTION DRAINING

- Allows existing connections to complete before a target is removed.
- Used during deployments, maintenance or scale-in events.
- Prevents abrupt termination of active sessions.
- Improves user experience and reduces errors.
- Configure an appropriate deregistration delay (e.g. 30-300 seconds).

7 STICKY SESSIONS

- Keeps a client's requests on the same server.
- Uses cookies, session IDs or source IP.
- Useful for stateful applications (e.g. user sessions).
- Can reduce load distribution efficiency.
- Prefer stateless designs when possible.
- In L7, usually configured with application cookies.

8 FAILURE BEHAVIOR

- Unhealthy targets are automatically removed.
- Traffic is routed to healthy targets.
- If all targets are unhealthy, clients may see errors (5xx).
- Design for multiple AZs and redundancy.
- Monitor health check failures and set up alerts.

9 BEST PRACTICES

- Use L7 for web applications and advanced routing.
- Use L4 for high performance and simple use cases.
- Configure proper health checks and thresholds.
- Enable connection draining.
- Use sticky sessions only when required.
- Design for multiple AZs and failure scenarios.
- Monitor metrics: latency, error rate, healthy targets, connections.

Redundancy, N+1 capacity, active-active, active-passive, quorum, eliminating single points of failure

1 WHAT IS HIGH AVAILABILITY (HA)

- Keeps services running even when components fail.
- Reduces downtime and maintains user access.
- Uses redundancy and automatic failover.
- Eliminates single points of failure (SPOF).
- Designed for failures, not perfect environments.



HA = REDUNDANCY
+ AUTOMATIC RECOVERY
+ NO SINGLE POINT
OF FAILURE

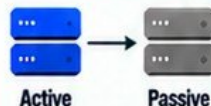
3 COMMON HA ARCHITECTURE PATTERNS

ACTIVE-ACTIVE



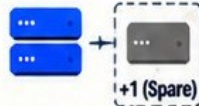
- Both sites handle traffic.
- Best for high availability and scalability.
- Requires data synchronization.
- More complex.

ACTIVE-PASSIVE



- One active, one standby.
- Standby takes over on failure.
- Simpler and lower cost.
- Common for databases and critical services.

N+1 CAPACITY



- Extra capacity ready to take over.
- Can be server, node or service.
- Improves resilience.
- Used for planned and unplanned failures.

4 QUORUM AND CONSENSUS (FOR DISTRIBUTED SYSTEMS)



- A quorum is the minimum number of nodes required to make decisions.
- Prevents split-brain (two sides thinking they are primary).
- Common in databases, storage systems and distributed services.
- Examples: etcd, PostgreSQL, MongoDB, Kafka.

6 FAILURE SCENARIOS AND RECOVERY

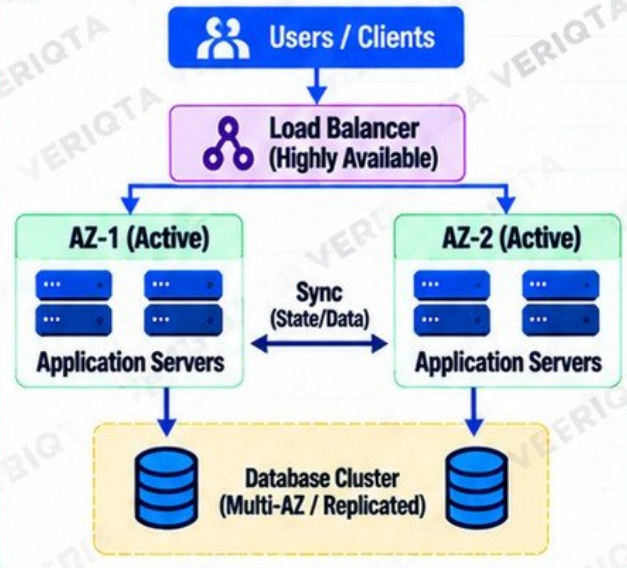
Component Failure	Example	HA Strategy	Result
Server failure	Application server down	N+1, auto-replace	Traffic continues
AZ failure	Entire availability zone down	Multi-AZ, active-active	Serves from other AZ
Database failure	Primary database down	Replication, failover	Automatic promotion
Load balancer failure	LB instance down	Multiple LB, health checks	Traffic routes to healthy LB
Region failure	Entire region unavailable	Multi-region, DNS failover	Traffic routes to other region
Dependency failure	External API down	Retries, failover, circuit breaker	Service degrades gracefully

8 BENEFITS OF HIGH AVAILABILITY



- Minimal downtime and higher reliability.
- Better user experience and trust.
- Handles expected and unexpected failures.
- Supports scaling and business growth.
- Increases operational resilience.

2 HIGH AVAILABILITY ARCHITECTURE OVERVIEW



5 ELIMINATING SINGLE POINTS OF FAILURE



REDUNDANT
ARCHITECTURE

- Remove single servers and single components.
- Use multiple AZs and regions where required.
- Redundant load balancers, application servers, databases and storage.
- Design for component and dependency failures.

7 KEY DESIGN CONSIDERATIONS



- Identify and remove all SPOFs.
- Use redundancy at multiple layers.
- Design for the right RTO and RPO.
- Plan for AZ and region failures.
- Use health checks and automatic recovery.
- Test failover regularly.
- Consider data consistency and quorum.
- Monitor and alert on failure scenarios.

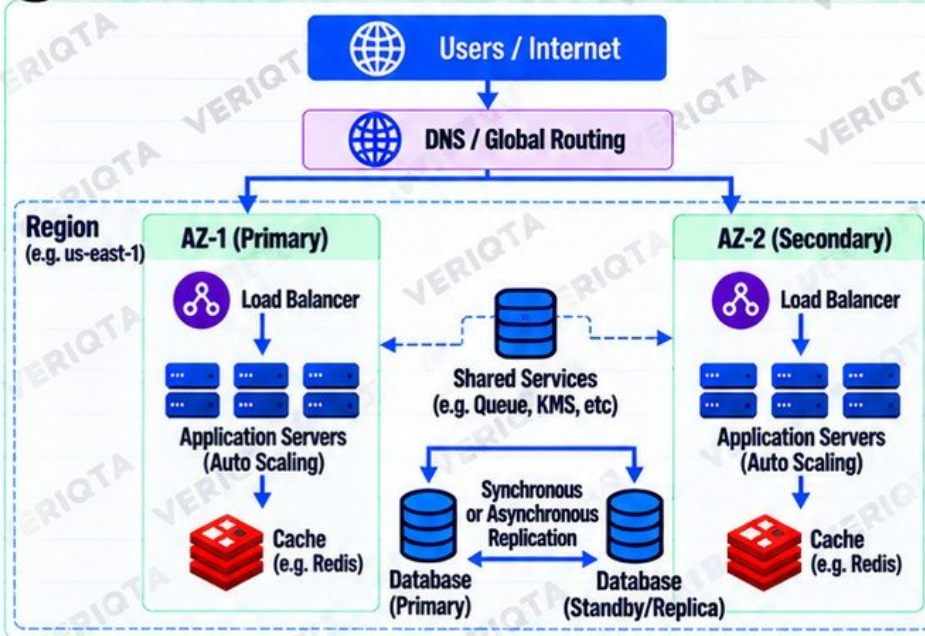
9 COMMON MISTAKES



- Relying on a single server or AZ.
- Not testing failover and recovery.
- Ignoring dependent services (DNS, APIs, databases).
- Misconfigured health checks.
- Assuming one layer of redundancy is enough.
- Not planning for data consistency and split-brain.

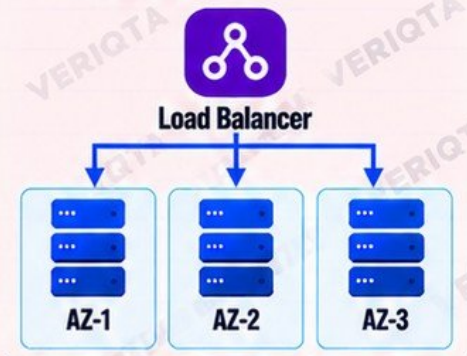
Workload distribution, data placement, cross-AZ dependencies, failover, latency and cost

1 MULTI-AZ ARCHITECTURE OVERVIEW



2 WORKLOAD DISTRIBUTION

- Distribute application instances across multiple AZs.
- Use load balancers to route traffic to healthy instances.
- Keep a minimum of one instance in each AZ.
- Use Auto Scaling to handle traffic spikes.
- Ensure stateless applications for easier distribution.



3 DATA PLACEMENT STRATEGIES

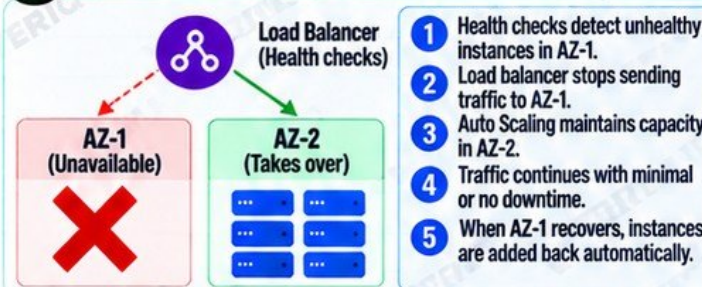
Data Type	Placement	Notes
Application data	Multi-AZ (replicated)	Use managed database with Multi-AZ.
Cache data	Multi-AZ (replication)	Use Redis with cross-AZ replication.
File storage	Regional service (e.g. S3, EFS)	Data available across AZs.
Message queues	Regional service (e.g. SQS, Kafka)	Automatically multi-AZ.
Configuration / secrets	Regional service (e.g. AWS Secrets Manager)	Highly available by design.
Logs and monitoring	Regional (e.g. CloudWatch)	Stored across multiple AZs.

4 CROSS-AZ DEPENDENCIES

- COMMON DEPENDENCIES**
- Application in one AZ depends on a database in another AZ.
 - Tight coupling between services across AZs.
 - A single NAT gateway for all AZs.
 - Cross-AZ synchronous calls without timeouts.
 - Shared state that is not replicated.
 - Single load balancer or DNS configuration.

- BEST PRACTICES**
- Keep dependencies regional and highly available.
 - Use replicated databases or managed services.
 - Use one NAT gateway per AZ.
 - Use timeouts, retries and circuit breakers.
 - Avoid single AZ dependencies.
 - Design for failure and test cross-AZ failures regularly.

5 FAILOVER SCENARIO



6 LATENCY AND COST CONSIDERATIONS

Aspect	Details	Impact
Cross-AZ latency	Typically 1–2 ms within a region	Low, but can affect synchronous calls
Data transfer cost	Cross-AZ traffic is charged by cloud provider	Increases cost for chatty services
Database replication	Synchronous = lower RPO, higher latency	Trade-off between consistency and performance
Service design	Minimize cross-AZ traffic where possible	Reduces cost and latency
User experience	Global routing + nearest region (if needed)	Better performance for global users

7 KEY DESIGN CONSIDERATIONS



- Which components must be in multiple AZs?
- How is data replicated and what is the RPO?
- What happens when an AZ fails?
- Are there any single points of failure?
- How much cross-AZ traffic is acceptable?
- What is the expected latency and cost impact?
- How will you test failover regularly?

8 SENIOR ENGINEER TAKEAWAYS

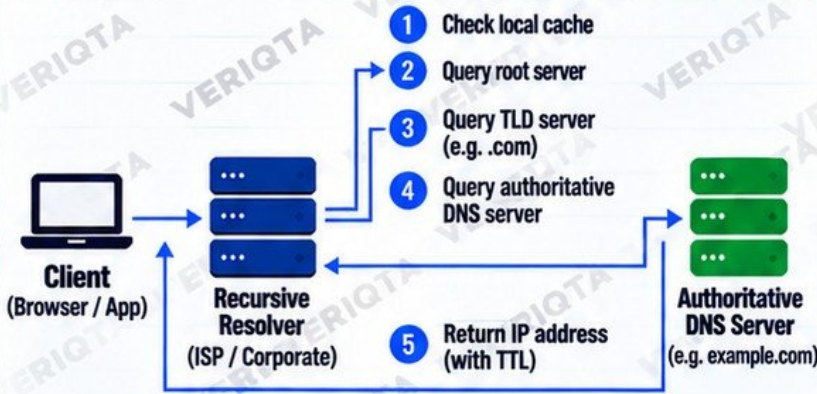


- Use at least two AZs for production workloads.
- Keep services stateless where possible.
- Use managed services with built-in multi-AZ support.
- Understand the trade-offs between latency, consistency and cost.
- Avoid single AZ dependencies and test failover regularly.
- Design for resilience, not just high availability.

08 DNS, Service Discovery & Routing

Resolution paths, TTLs, caching, discovery, failover assumptions and DNS-related outages

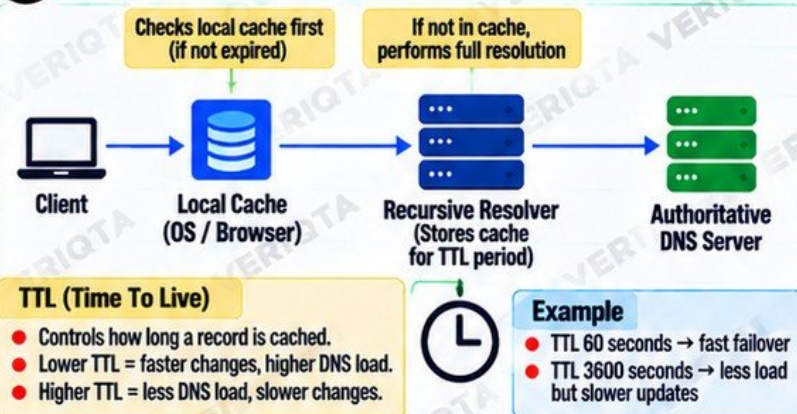
1 DNS RESOLUTION PATH (EXAMPLE)



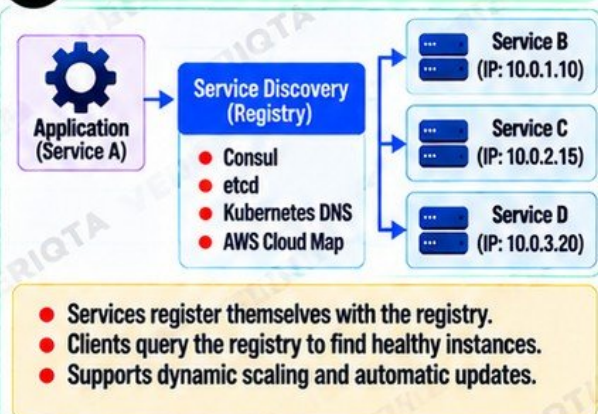
2 DNS RECORD TYPES (COMMON)

Record Type	Purpose	Example
A	Maps domain to IPv4 address	example.com → 1.2.3.4
AAAA	Maps domain to IPv6 address	example.com → 2001:db8::1
CNAME	Alias to another domain	www → example.com
MX	Mail server routing	example.com → mail.example.com
NS	Authoritative name servers	example.com → ns1.example.com
TXT	Stores text (SPF, DKIM, etc)	example.com → "v=spf1 ..."
SRV	Service discovery for applications	_sip_tcp → sip.example.com

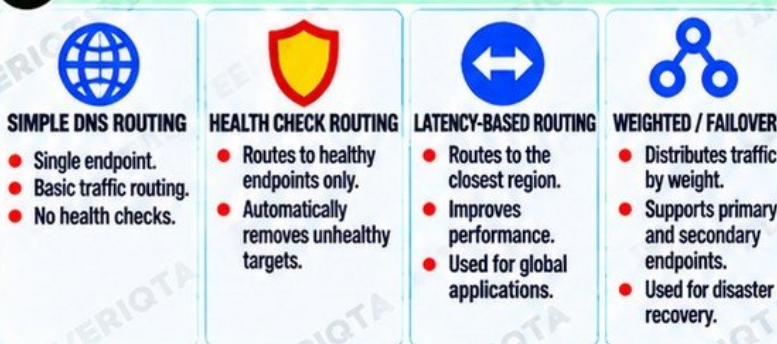
3 DNS CACHING AND TTL



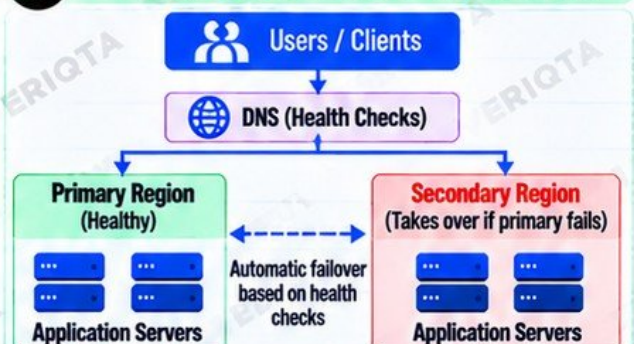
4 SERVICE DISCOVERY (INTERNAL)



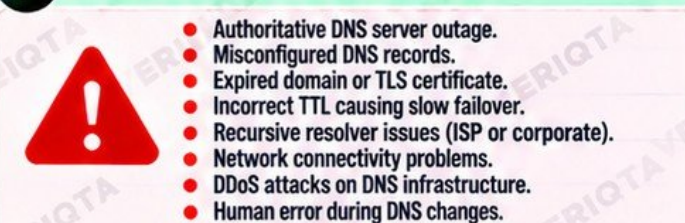
5 ROUTING METHODS



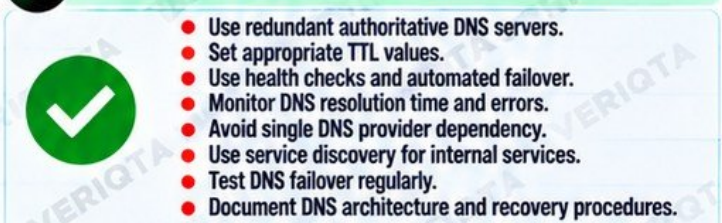
6 DNS FAILOVER ARCHITECTURE (EXAMPLE)



7 COMMON DNS-RELATED OUTAGES



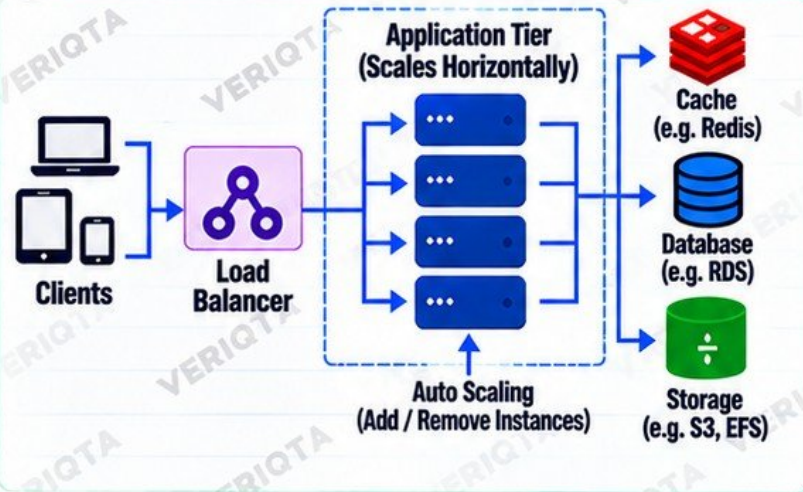
8 BEST PRACTICES



09 Application Scaling Architecture

Horizontal vs vertical scaling, autoscaling signals, bottlenecks, cold starts, scaling dependencies

1 SCALING ARCHITECTURE OVERVIEW



2 HORIZONTAL VS VERTICAL SCALING

HORIZONTAL SCALING (Scale Out)



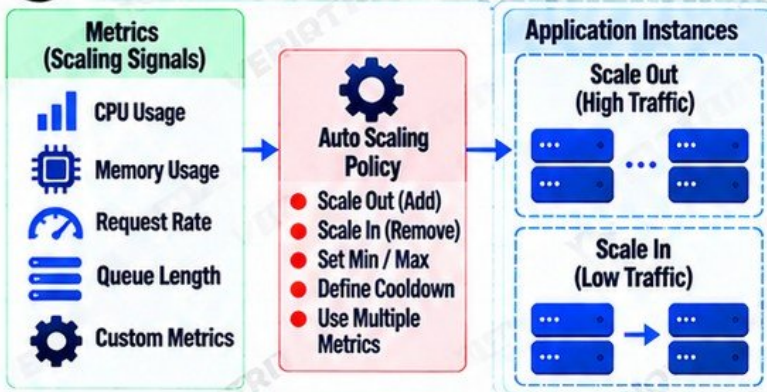
- Add more instances/servers
- Handles more traffic
- Better fault tolerance
- Common in cloud and container environments
- Used with load balancers and autoscaling
- More cost efficient at scale

VERTICAL SCALING (Scale Up)



- Increase CPU, memory or storage on a single server
- Simple to implement
- Limited by hardware
- Creates a single point of failure
- May require downtime for resizing
- Not suitable for very high traffic workloads

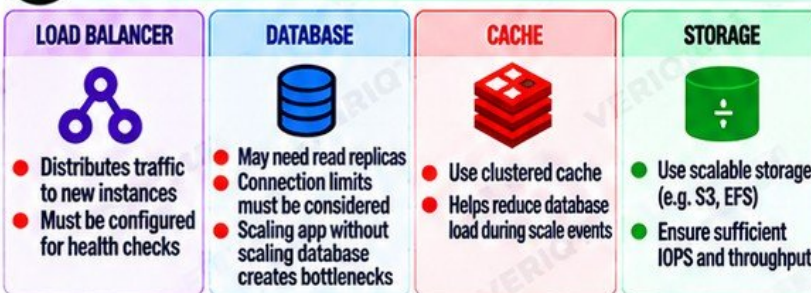
3 AUTOSCALING ARCHITECTURE



4 COMMON SCALING SIGNALS

Signal	Description	When to Scale Out
CPU Usage	High CPU on instances	CPU consistently above threshold (e.g. 70%)
Memory Usage	High memory consumption	Memory above threshold
Request Rate (RPS)	Increased user traffic	RPS above threshold
Queue Length	Growing background jobs	Queue length increasing
Custom Metrics	Application specific (e.g. active users)	Metric above threshold

5 SCALING DEPENDENCIES



6 COLD STARTS AND WARM STRATEGIES



COLD STARTS

- Time taken to start a new instance or container.
- Can cause temporary higher latency.
- Common in serverless and container environments.
- Affected by image size, initialization scripts and dependencies.



WARM STRATEGIES

- Use lightweight images.
- Pre-initialize important components.
- Keep a minimum number of warm instances.
- Use faster startup configurations.
- Optimize dependency startup (e.g. database connections).

7 COMMON BOTTLENECKS DURING SCALING

Component	Bottleneck	Impact	Solution
Database	Connection limits	High latency, errors	Use read replicas, pooling
Cache	Insufficient capacity	More DB load	Use clustered cache
Network	Bandwidth limits	Slow responses	Use scalable network, CDN
Storage	IOPS / throughput	Slow read/write	Use scalable storage
Application	Inefficient code	High CPU / memory	Optimize code, profile
Dependencies	External API limits	Timeouts, failures	Use retries, queue, caching

8 BEST PRACTICES



- Use horizontal scaling for high traffic workloads.
- Set clear autoscaling policies and thresholds.
- Monitor all dependencies (database, cache, storage).
- Configure proper health checks.
- Test scaling behavior regularly.
- Plan for database and dependency scaling.
- Optimize for fast startup to reduce cold starts.
- Monitor cost and avoid over-provisioning.
- Use load testing to validate scaling limits.
- Have alerts for scaling failures.

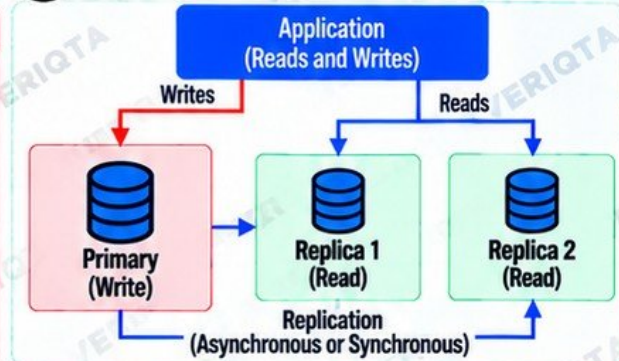
10 Database Architecture Decisions

SQL vs NoSQL considerations, primary/replica patterns, consistency, connection limits, failover and contention

1 SQL VS NOSQL CONSIDERATIONS

SQL (Relational)	NoSQL (Non-Relational)
- Structured data (tables) - Strong consistency (ACID) - Complex queries (JOIN) - Best for transactional systems - Examples: PostgreSQL, MySQL, SQL Server, Oracle	- Flexible data models - Designed for horizontal scaling - High availability and large data - Eventual consistency (in many cases) - Best for high-scale and unstructured data - Examples: MongoDB, Cassandra, DynamoDB, Redis, Elasticsearch

2 PRIMARY / REPLICA ARCHITECTURE (SQL EXAMPLE)



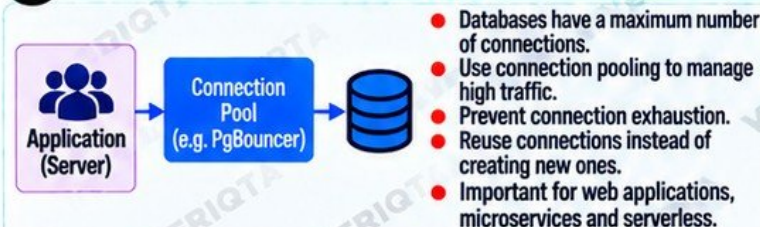
3 COMMON DATABASE ARCHITECTURE PATTERNS

SINGLE NODE	PRIMARY / REPLICA	MULTI-AZ (HA)	SHARDED / DISTRIBUTED
- Simple setup - No redundancy - Suitable for development or small workloads	- One primary, multiple read replicas - Improves read performance - Used for high availability and scaling	- Replicates data across AZs - Automatic failover - High availability - Reduces regional downtime risk	- Data split across shards - Horizontal scaling - Handles very large datasets - More complex architecture - Common in NoSQL systems

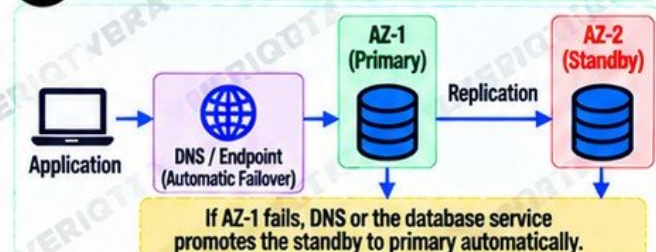
4 CONSISTENCY MODELS

STRONG CONSISTENCY	- All reads get the latest write. - Used in most SQL databases.
EVENTUAL CONSISTENCY	- Updates propagate over time. - Common in NoSQL and distributed systems.
READ AFTER WRITE	- User sees their own writes immediately. - Often used in session-based systems.
TUNABLE CONSISTENCY	- Choose consistency vs availability. - Common in systems like Cassandra.

5 CONNECTION LIMITS AND POOLING



6 FAILOVER ARCHITECTURE (MULTI-AZ EXAMPLE)



7 COMMON BOTTLENECKS AND CONTENTION

Bottleneck	Cause	Impact	Mitigation
CPU	Complex queries	Slow queries	Optimize queries, add indexes
Memory	Large datasets in memory	High latency, crashes	Increase memory, optimize queries
Disk I/O	High write/read load	Slow responses	Use faster storage, optimize queries
Connection limits	Too many clients	Connection failures	Use connection pooling
Lock contention	Competing writes	Slow transactions	Reduce lock scope, use indexing
Replication lag	High write volume	Stale reads	Scale replicas, optimize writes
Network	Cross-region access	Higher latency	Keep services in same region/AZ

8 SQL VS NOSQL USE CASES

SQL (Use When)	NoSQL (Use When)
- You need transactions (ACID). - You have structured data. - You need complex queries (JOIN). - Data consistency is critical. - Examples: banking, e-commerce, enterprise systems.	- You need to scale horizontally. - You have unstructured or semi-structured data. - You can tolerate eventual consistency. - You need very high availability. - Examples: real-time analytics, logging, content platforms.

9 BEST PRACTICES

-
- Choose the right database for the workload.
 - Use read replicas for read-heavy workloads.
 - Enable Multi-AZ for high availability.
 - Use connection pooling.
 - Monitor performance (CPU, memory, IOPS, slow queries).
 - Plan for failover and recovery regularly.
 - Test under real load.

10 KEY TAKEAWAYS

-
- There is no single best database for everything.
 - Understand consistency, availability and scalability trade-offs.
 - Design for failures, not just normal operations.
 - Monitor and manage connection limits.
 - Use replicas, Multi-AZ and backups for high availability.
 - Be aware of bottlenecks and contention.
 - Test failover, recovery and scaling regularly.

11 Caching Architecture

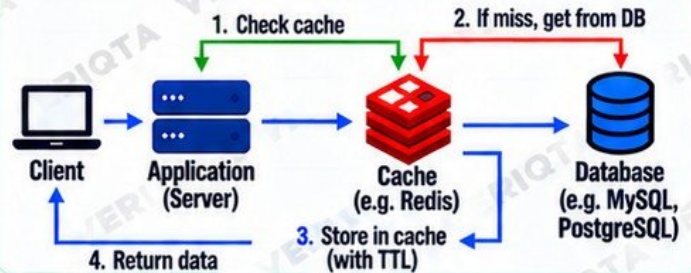
Cache-aside, write-through concepts, TTLs, invalidation, stampedes, stale data and cache failure

1 WHAT IS CACHING?

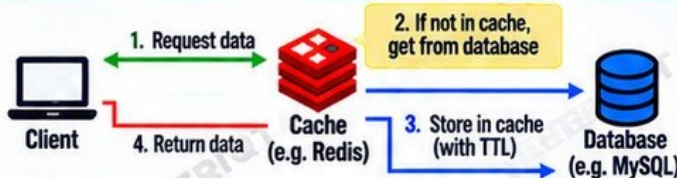
- Stores frequently accessed data in fast storage (memory).
- Reduces latency and database load.
- Improves application performance and scalability.
- Can be in-memory (e.g. Redis, Memcached) or at multiple layers (CDN, application, database).



2 CACHING ARCHITECTURE OVERVIEW



3 CACHE-ASIDE (LAZY LOADING)



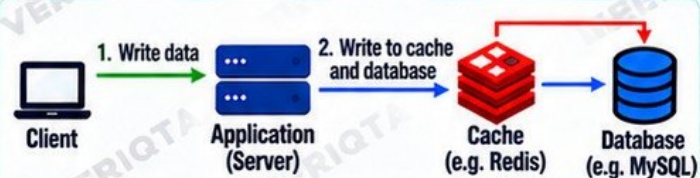
HOW IT WORKS

- Application checks cache first.
- If data is not found (cache miss), it reads from the database.
- Stores the data in cache with a TTL.
- Returns the data to the client.

USE CASES

- Read-heavy workloads.
- Data that can be cached temporarily.
- Common in web applications and APIs.

4 WRITE-THROUGH CACHING



HOW IT WORKS

- Application writes data to cache and database at the same time.
- Cache is always up to date.
- Simpler to manage consistency.
- Higher write latency (because it writes to both).

USE CASES

- Data that must always be consistent.
- Systems with frequent reads after writes.
- Common in enterprise applications.

5 TTL AND EXPIRATION

HOW TTL WORKS

- Each cached item has a time to live (TTL).
- After the TTL expires, the item is removed from cache.
- Next request will fetch fresh data from the database.

EXAMPLE

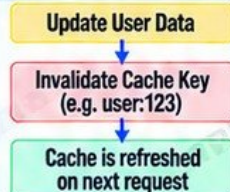
Key: user:123
TTL: 3600 seconds (1 hour)



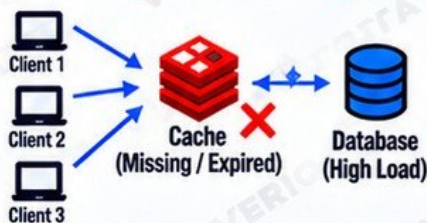
6 CACHE INVALIDATION

- Remove or update cached data when the source data changes.
- Types:
 - Time-based (TTL).
 - Event-based (on write/update).
 - Manual invalidation (admin or script).
- Avoid serving stale data after updates.

INVALIDATION EXAMPLE



7 CACHE STAMPEDE (THUNDERING HERD)



WHAT HAPPENS

- Many clients request the same key at the same time after expiry.
- Cache is empty.
- All requests go to the database.
- Database gets overloaded.

MITIGATION

- Use longer TTLs.
- Add jitter to TTL.
- Use request coalescing (single flight).
- Use stale-while-revalidate.

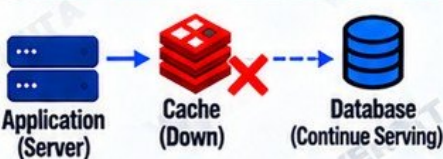
8 STALE DATA

- Cached data may be outdated.
- Acceptable for non-critical data (e.g. product listings, news).
- Not acceptable for critical data (e.g. payments, account balances).
- Use short TTLs or event-based invalidation for critical data.

EXAMPLES

- Product catalog: 5-30 minutes (stale data acceptable).
- User balance: real-time or very short TTL (stale data not acceptable).

9 CACHE FAILURE AND RESILIENCE



HANDLING CACHE FAILURE

- Application should continue working (fallback to database).
- Use circuit breakers.
- Monitor cache health.
- Have alerts and auto-recovery.
- Design for graceful degradation (higher latency, but still functional).

10 BEST PRACTICES

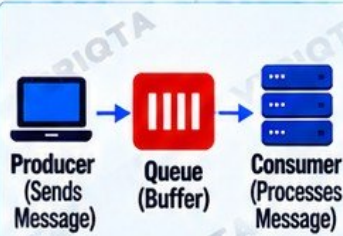
- Cache the right data (high read, low change).
- Use appropriate TTLs.
- Implement proper invalidation strategies.
- Prevent cache stampedes.
- Monitor cache hit/miss ratio.
- Plan for cache failure and recovery.
- Use multi-layer caching (CDN, application, database).
- Test caching behavior under load.
- Keep critical data consistent.

12 Queues & Asynchronous Systems

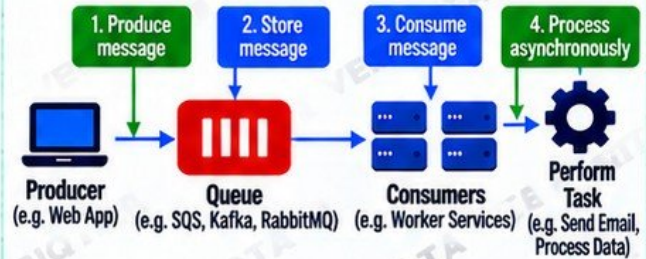
Producers, consumers, backpressure, retries, dead-letter queues, idempotency and backlog growth

1 WHAT ARE QUEUES?

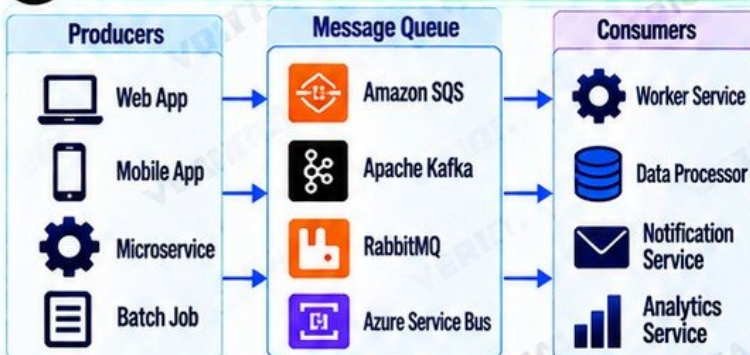
- A queue is a buffer that stores messages for asynchronous processing.
- Producers send messages to the queue.
- Consumers process messages at their own pace.
- Decouples services, improves reliability and resilience.
- Common in microservices, event-driven systems and background jobs.



2 HOW IT WORKS (FLOW)



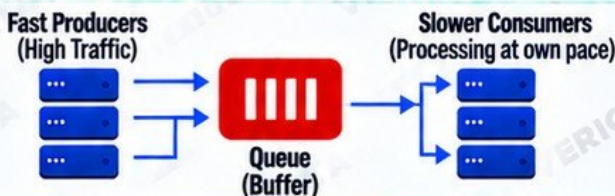
3 COMMON QUEUE ARCHITECTURE



4 KEY CONCEPTS

Concept	Description
Producer	Sends messages to the queue.
Consumer	Reads and processes messages.
Message	Unit of data containing the task or event.
Asynchronous	Producer and consumer do not wait for each other.
Decoupling	Services are independent and more resilient.
Scalability	Consumers can scale independently.
Durability	Messages are stored safely until processed.
Ordering	Some systems guarantee order (e.g. FIFO queues).

5 BACKPRESSURE AND FLOW CONTROL



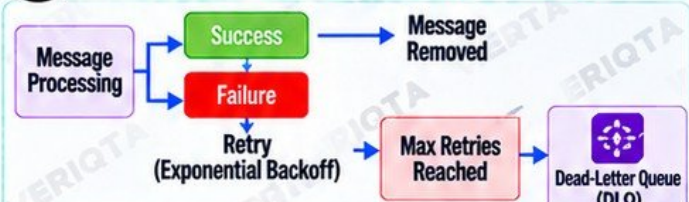
WHAT HAPPENS?

- Producer sends messages faster than consumers can process.
- Queue size grows (backlog).
- If not controlled, it can lead to high latency, high cost or failure.

SOLUTIONS

- Monitor queue size (depth).
- Apply rate limits.
- Use backpressure mechanisms.
- Scale consumers.
- Set proper autoscaling policies.

6 RETRIES AND DEAD-LETTER QUEUES (DLQ)



BENEFITS

- Retries handle temporary failures.
- Exponential backoff reduces pressure on dependencies.
- DLQ stores messages that repeatedly fail.
- Helps with debugging and manual reprocessing.

COMMON USE CASES

- Network timeouts.
- Downstream service failures.
- Invalid or corrupted data.
- Third-party API failures.
- Permanent errors after retries.

7 IDEMPOTENCY

- Idempotency means processing the same message multiple times has the same result as processing it once.
- Important because retries can cause duplicate message delivery.
- Use unique message IDs, idempotency keys or database constraints.



8 BACKLOG GROWTH AND MONITORING

- Queue size (depth) increases when consumers are slower than producers.
- Can lead to higher latency, timeouts and increased costs.
- Monitor queue metrics and set alerts.

KEY METRICS

- Queue size (messages).
- Oldest message age.
- Message processing rate.
- Send/receive error rate.
- Consumer lag (Kafka).

9 COMMON USE CASES



10 BEST PRACTICES

- Design idempotent consumers.
- Use retries with exponential backoff.
- Implement dead-letter queues.
- Monitor queue size and processing time.
- Set alerts for backlog growth and failures.
- Scale consumers based on workload.
- Test failure scenarios regularly.
- Keep messages small and well-structured.

13 Resilient Dependency Design

Timeouts, retries, exponential backoff, jitter, circuit breakers, bulkheads and cascading failures

1 WHY RESILIENT DEPENDENCIES MATTER

- Modern applications depend on many external services (databases, APIs, third-party).
- Failures and slowness will happen.
- Without resilience, one failing service can bring down the entire system.
- Use timeouts, retries, circuit breakers, bulkheads and backoff to keep your system stable and available.



2 COMMON DEPENDENCY FAILURE MODES

SLOW RESPONSE



- High latency
- Network issues
- Resource limits

TIMEOUT



- Service not responding
- Network congestion
- Overloaded service

ERRORS



- 5xx errors
- Connection failures
- Unexpected responses

COMPLETE OUTAGE



- Service down
- DNS failure
- Network partition
- Third-party outage

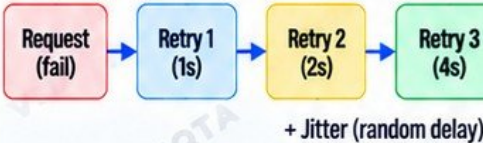
3 TIMEOUTS

- Set a timeout for every external call.
- Do not wait forever.
- Choose a timeout based on expected response time and your user experience.
- Always handle timeout errors gracefully.



4 RETRIES WITH EXPONENTIAL BACKOFF

- Retry failed requests (for temporary failures).
- Use exponential backoff (increasing delay).
- Add jitter (randomness) to avoid retry storms.
- Do not retry for all errors (e.g. 4xx).



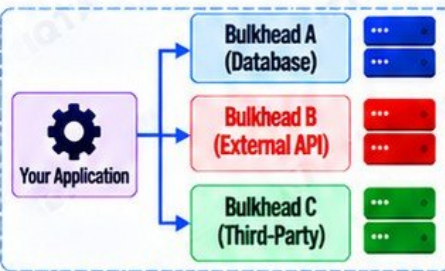
5 CIRCUIT BREAKER

- Stops calling a failing service to prevent cascading failures.
- Has three states: Closed, Open, Half-Open.
- Allows the service to recover over time.



6 BULKHEADS

- Isolate resources for each dependency.
- Prevent one failing service from consuming all resources.
- Use separate thread pools, connection pools or containers.



7 CASCADING FAILURES

- A failure in one service can cause other services to fail.
- Happens when resources are exhausted (threads, connections, CPU, etc).
- Use timeouts, retries, circuit breakers and bulkheads to prevent this.



8 RESILIENT DEPENDENCY FLOW



9 KEY CONSIDERATIONS

Aspect	What to Consider
Timeouts	Set based on expected response time and SLA.
Retries	Use exponential backoff and jitter. Avoid retrying 4xx.
Circuit Breaker	Protects system from repeated failures.
Bulkheads	Isolate resources per dependency.
Idempotency	Ensure operations can be retried safely.
Monitoring	Track latency, error rates and circuit breaker state.

10 BEST PRACTICES



- Set appropriate timeouts for all external calls.
- Use exponential backoff with jitter for retries.
- Implement circuit breakers for unstable services.
- Use bulkheads to isolate dependencies.
- Design idempotent operations.
- Monitor dependency health and set alerts.
- Plan for graceful degradation and fallback.
- Test failure scenarios regularly (chaos testing).

11 COMMON USE CASES

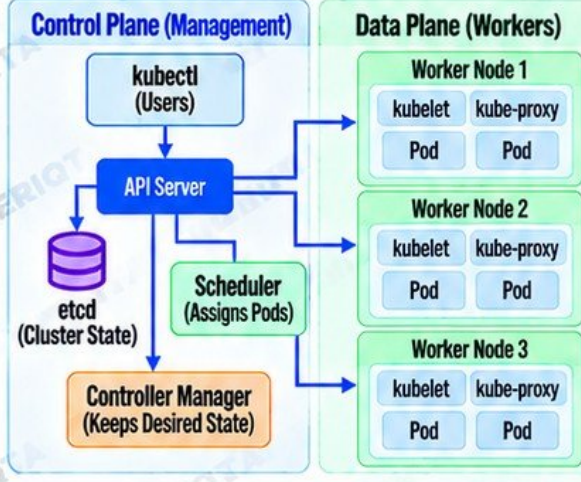


14 Kubernetes Production Architecture

Control/data plane boundaries, ingress, services, workloads, nodes, scheduling, disruption and failure domains

1 KUBERNETES ARCHITECTURE OVERVIEW

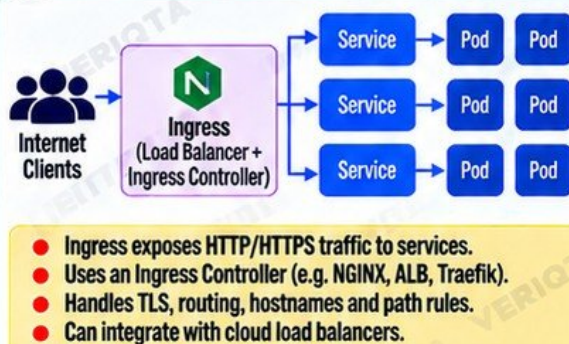
- Kubernetes has a control plane that manages the cluster and a data plane where workloads run.
- It provides scheduling, self-healing, scaling and high availability.
- Components communicate over a secure API.
- Designed for resilient, scalable and automated production environments.



2 CONTROL PLANE vs DATA PLANE

CONTROL PLANE (MANAGEMENT)	DATA PLANE (WORKERS)
• API Server (entry point) • etcd (cluster state) • Scheduler (assigns pods) • Controller Manager (controllers) • Manages desired state • Handles scaling, healing and updates • Should be highly available (multiple replicas)	• Worker nodes run application workloads • kubelet (manages pods) • kube-proxy (networking) • Runs containers (pods) • Handles networking and service traffic • Can be added or removed for scaling • Workloads are distributed across multiple nodes/AZs

3 INGRESS ARCHITECTURE



- Ingress exposes HTTP/HTTPS traffic to services.
- Uses an Ingress Controller (e.g. NGINX, ALB, Traefik).
- Handles TLS, routing, hostnames and path rules.
- Can integrate with cloud load balancers.

4 SERVICES AND NETWORKING

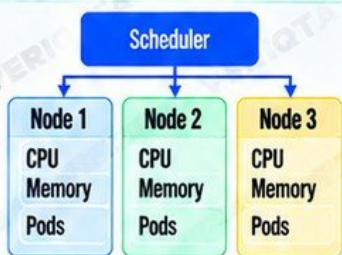
- Services provide a stable network endpoint for pods.
- Types: ClusterIP, NodePort, LoadBalancer, ExternalName.
- kube-proxy handles traffic routing (using iptables or IPVS).
- Services enable load balancing between healthy pods.
- Network policies control traffic between pods.
- DNS is used for service discovery (e.g. myapp.default.svc.cluster.local).

5 WORKLOADS IN KUBERNETES



6 NODES AND SCHEDULING

- Scheduler places pods on the best node based on resources, constraints and rules.
- Considers CPU, memory, taints, tolerations, affinity and anti-affinity.
- Nodes can be physical or virtual (cloud).
- Use multiple nodes and multiple availability zones for high availability.
- Monitor node health and capacity.



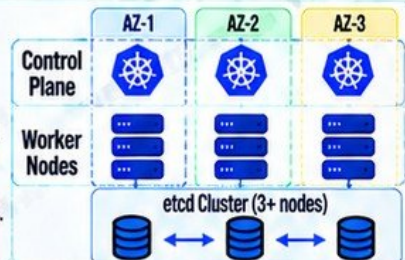
7 DISRUPTION AND FAILURE DOMAINS

- Failures can happen at different levels: pod, node, AZ, region or control plane.
- Use multiple nodes and AZs to reduce risk.
- Pods should be designed to be disposable.
- Use readiness and liveness probes.
- Configure Pod Disruption Budgets (PDB).
- Monitor cluster health and alerts.
- Plan for node drains, upgrades and auto-recovery.

- COMMON DISRUPTIONS**
- Pod crashes
 - Node failure
 - AZ/zone outage
 - Network partition
 - Control plane failure
 - Resource exhaustion
 - Rolling updates
 - Manual maintenance

8 HIGH AVAILABILITY ARCHITECTURE

- Run multiple control plane instances (e.g. 3+).
- Distribute worker nodes across multiple AZs.
- Use a highly available etcd cluster.
- Spread workloads using topology spread constraints.
- Use multiple replicas for applications.
- Test failure scenarios regularly.



9 BEST PRACTICES

- Use multiple AZs and nodes.
- Keep control plane highly available.
- Use readiness and liveness probes.
- Set resource requests and limits.
- Use HPA (Horizontal Pod Autoscaler).
- Use PDB to manage disruptions.
- Implement network policies.
- Monitor cluster health and alerts.
- Plan for upgrades and node drains.
- Test failure scenarios.

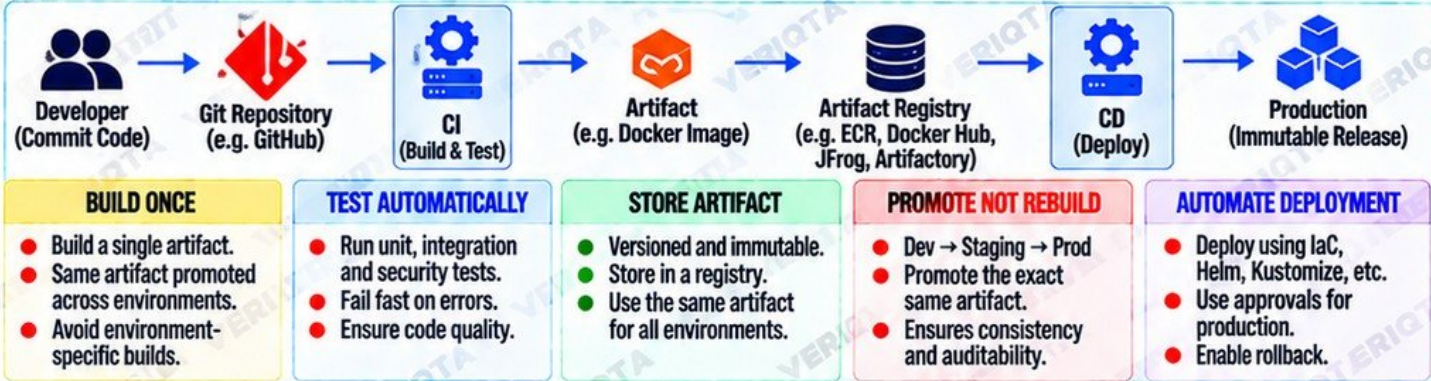
10 COMMON USE CASES

- Web Applications (nginx, app servers)
- Microservices (Containerized apps)
- Stateful Applications (databases, queues)
- Batch and Scheduled Jobs
- Cloud-Native Platforms

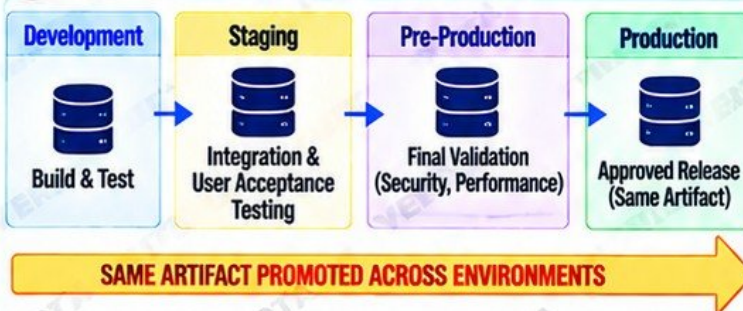
15 CI/CD & Release Architecture

Build once, artifact promotion, immutable releases, approvals, canary, blue-green, rollback and deployment safety

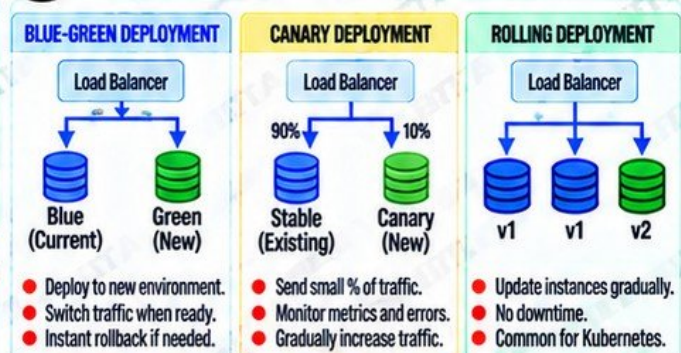
1 CI/CD PIPELINE OVERVIEW



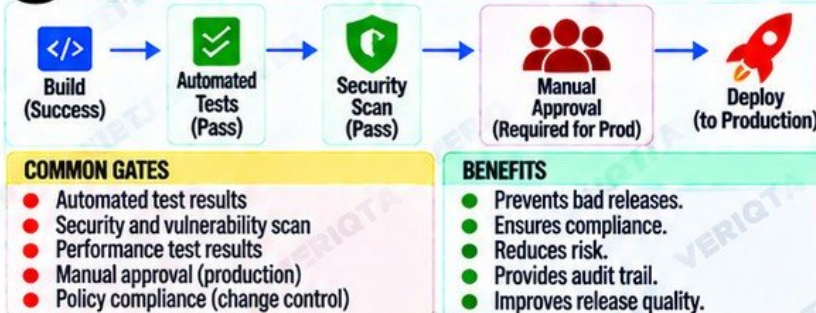
2 ENVIRONMENT PROMOTION FLOW



3 DEPLOYMENT STRATEGIES



4 APPROVALS AND GATES



5 IMMUTABLE RELEASES



6 ROLLBACK STRATEGY



7 DEPLOYMENT SAFETY PRACTICES



8 COMMON CI/CD TOOLS



9 KEY TAKEAWAYS



10 COMMON USE CASES

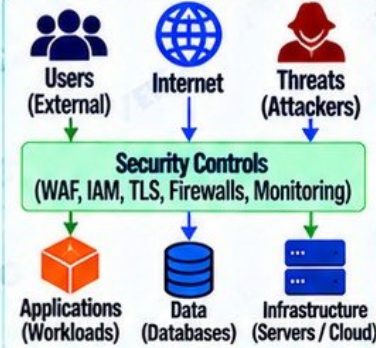


16 Security Architecture

Trust boundaries, IAM, workload identity, secrets, TLS, network segmentation, least privilege and auditability

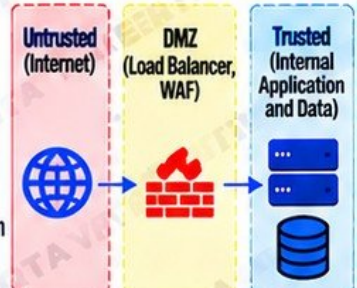
1 SECURITY ARCHITECTURE OVERVIEW

- Protects applications, data, users and infrastructure.
- Defines trust boundaries and access controls.
- Uses defense in depth (multiple layers).
- Applies least privilege, secure identity, network segmentation and continuous monitoring.
- Designed to prevent, detect and respond to threats.



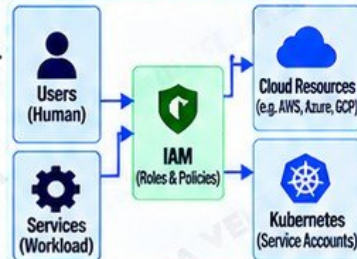
2 TRUST BOUNDARIES

- Separate different security zones (e.g. internet, public, private).
- Control how traffic and data move between zones.
- Reduce the impact of a breach.
- Enforce authentication, authorization and inspection at boundaries.



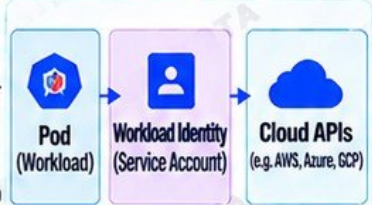
3 IDENTITY AND ACCESS MANAGEMENT (IAM)

- Manages who can access what.
- Use roles and policies (least privilege).
- Use temporary credentials where possible.
- Separate human and service accounts.
- Regularly review and audit permissions.



4 WORKLOAD IDENTITY

- Gives applications and workloads their own identity.
- Avoids using long-term credentials.
- Uses service accounts, managed identities or workload identity federation.
- Enforces least privilege access to cloud resources.



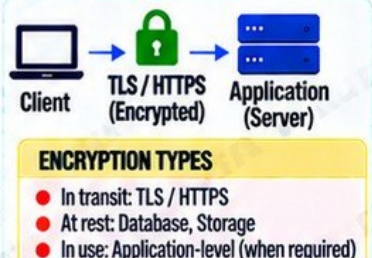
5 SECRETS MANAGEMENT

- Store sensitive data securely (e.g. API keys, database passwords).
- Do not hardcode secrets in code.
- Use a secrets manager (e.g. AWS Secrets Manager, Azure Key Vault, HashiCorp Vault).
- Rotate secrets regularly.
- Control access with least privilege.



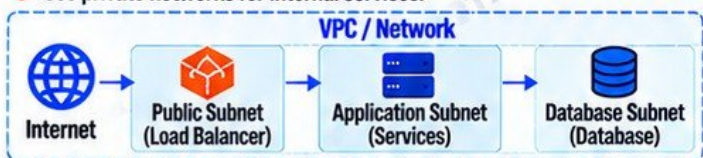
6 TLS AND ENCRYPTION

- Use TLS to encrypt data in transit (e.g. HTTPS, mTLS).
- Use strong cipher suites and regular certificate rotation.
- Encrypt sensitive data at rest (e.g. databases, object storage).
- Validate and manage certificates properly.



7 NETWORK SEGMENTATION

- Separate networks into different security zones.
- Use firewalls, security groups and network policies.
- Allow only required traffic (least privilege).
- Isolate critical systems (e.g. databases).
- Use private networks for internal services.



8 LEAST PRIVILEGE ACCESS

- Give only the minimum permissions required.
- Reduce the attack surface.
- Use role-based access control (RBAC).
- Remove unused permissions.
- Review and audit access regularly.



9 AUDITABILITY AND MONITORING

- Login and access logs.
- API and administrative actions.
- Changes to security policies and permissions.
- Use centralized logging and monitoring.
- Set up alerts for suspicious activity.



10 BEST PRACTICES

- Define clear trust boundaries.
- Use strong IAM and workload identities.
- Store and rotate secrets securely.
- Enforce TLS and encryption.
- Implement network segmentation.
- Apply least privilege access.
- Enable logging, monitoring and auditability.
- Regularly test and review security controls.



17 Observability Architecture

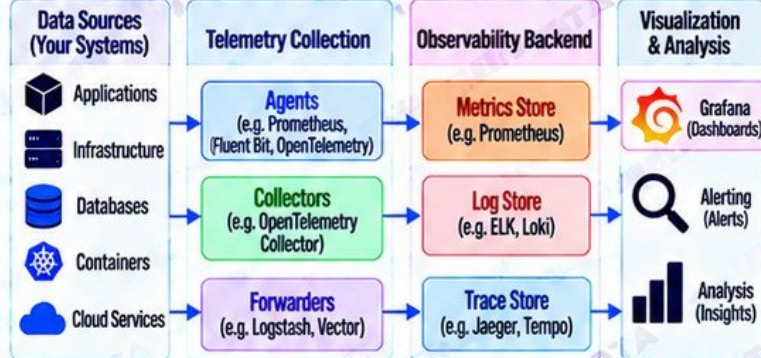
Metrics, logs, traces, events, telemetry pipelines, cardinality, correlation and actionable signals

1 WHAT IS OBSERVABILITY?

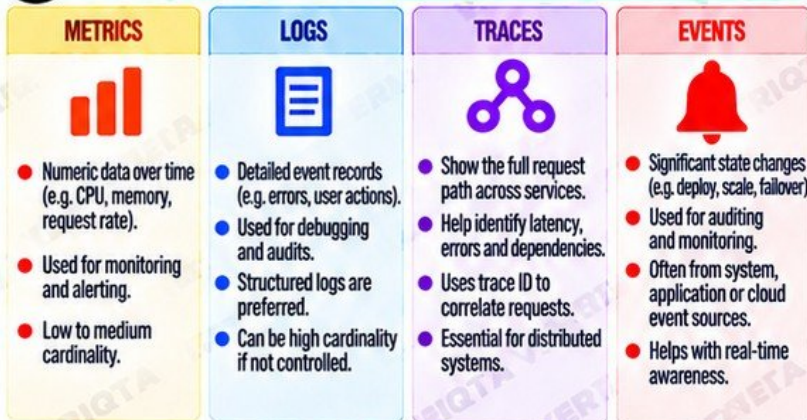
- Observability gives you deep visibility into how your system is behaving.
- Uses metrics, logs, traces and events to understand, troubleshoot and improve systems.
- Helps detect issues early, reduce MTTR and make better decisions.
- Essential for reliable, scalable and modern systems.



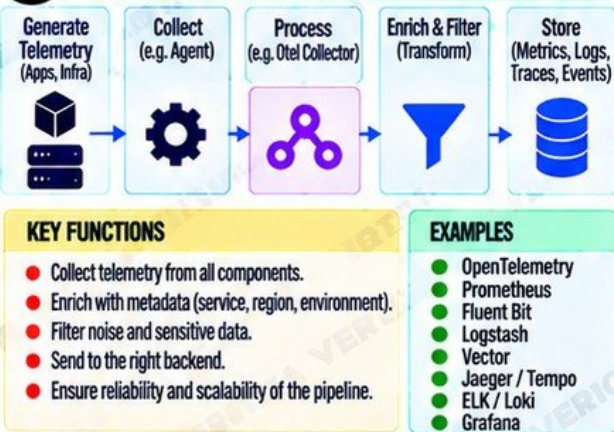
2 OBSERVABILITY ARCHITECTURE OVERVIEW



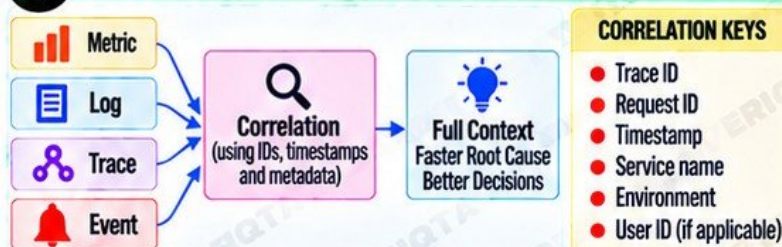
3 THE FOUR SIGNALS



4 TELEMETRY PIPELINE



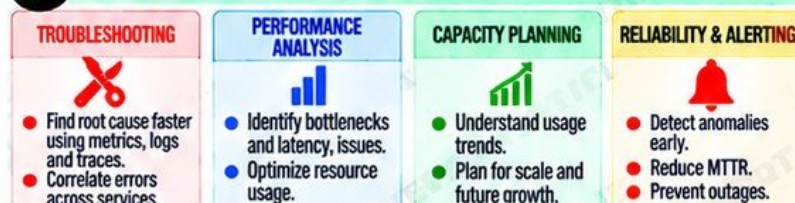
5 CORRELATION BETWEEN SIGNALS



6 CARDINALITY AND LABELS



7 OBSERVABILITY USE CASES



8 BEST PRACTICES

- Instrument all services with consistent telemetry.
- Use OpenTelemetry for traces, metrics and logs.
- Keep labels low cardinality and meaningful.
- Set up dashboards, alerts and SLOs.
- Monitor the observability pipeline itself.
- Use structured logs (JSON).
- Correlate signals using trace IDs and timestamps.
- Automate alerting for actionable signals (not noise).
- Regularly review and tune retention and costs.

9 COMMON TOOLS



10 COMMON MISTAKES

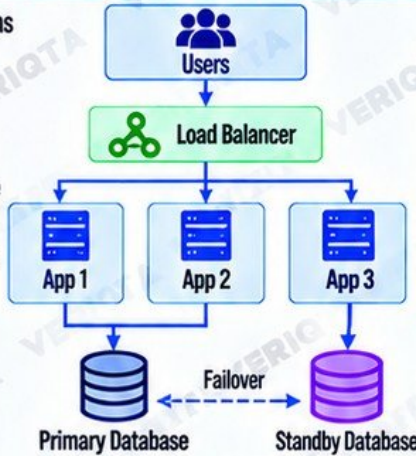
- No correlation between metrics, logs and traces.
- High cardinality labels.
- Alerting on noise instead of actionable signals.
- Missing instrumentation in services.
- Storing sensitive data in logs.
- Not monitoring the observability pipeline.
- Too short or too long retention.
- No clear ownership of dashboards and alerts.

18 Reliability & Disaster Recovery

SLI/SLO thinking, RTO, RPO, backups, restore verification, failover, regional recovery and dependency planning

1 RELIABILITY OVERVIEW

- Reliability ensures systems work as expected and recover from failures.
- Combines prevention, detection and recovery.
- Uses SLI/SLO to measure and manage reliability.
- Includes backups, failover, disaster recovery and dependency planning.



2 SLI, SLO AND SLA

SLI (Service Level Indicator)



- A measurable metric (e.g. availability, latency, error rate).
- Tells you how the system is performing.
- Examples: 99.9% availability, 200ms latency, <1% errors.

SLO (Service Level Objective)



- The target value for an SLI.
- Defines the expected level of reliability (e.g. 99.9% uptime).
- Used for operational decisions and error budgets.

SLA (Service Level Agreement)



- A formal agreement (often with customers).
- Defines consequences if SLO is not met.
- Usually less precise and includes business terms and penalties.

3 RTO AND RPO

RTO (Recovery Time Objective)



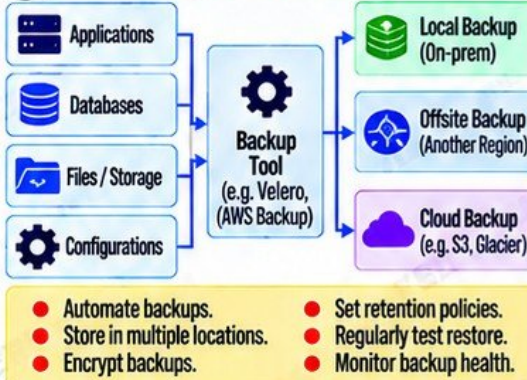
- How quickly you must restore the service after a failure.
- Example: RTO = 1 hour (service must be back within 1 hour).

RPO (Recovery Point Objective)



- How much data you can afford to lose.
- Example: RPO = 15 minutes (you can lose at most 15 minutes of data).

4 BACKUP ARCHITECTURE



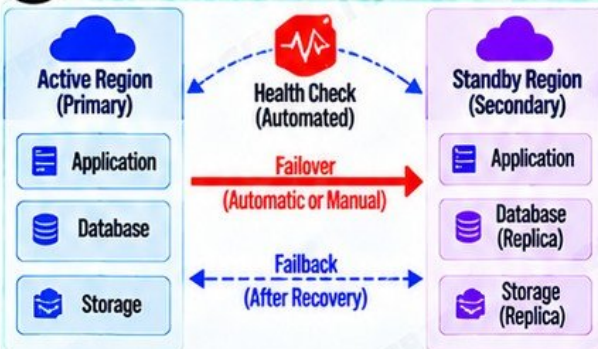
- Automate backups.
- Store in multiple locations.
- Encrypt backups.
- Set retention policies.
- Regularly test restore.
- Monitor backup health.

5 RESTORE VERIFICATION

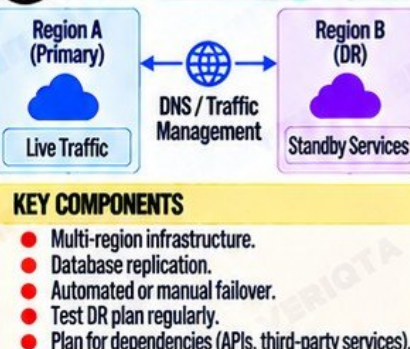


- Regularly test that backups can be restored.
- Verify data integrity and application functionality.
- Test both full and partial restores.
- Document the restore process.
- Automate restore tests where possible.

6 FAILOVER ARCHITECTURE



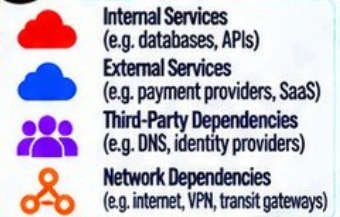
7 REGIONAL DISASTER RECOVERY



KEY COMPONENTS

- Multi-region infrastructure.
- Database replication.
- Automated or manual failover.
- Test DR plan regularly.
- Plan for dependencies (APIs, third-party services).

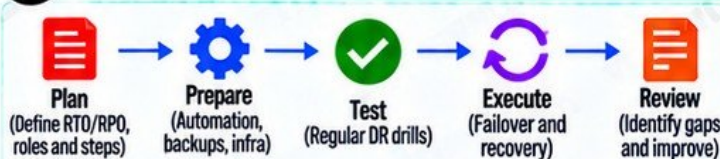
8 DEPENDENCY PLANNING



FOR EACH DEPENDENCY

- Identify criticality.
- Understand failure modes.
- Have a fallback or alternative.
- Include in DR testing.

9 DISASTER RECOVERY PLAN



10 BEST PRACTICES



- Define clear SLI/SLO, RTO and RPO.
- Automate and encrypt backups.
- Test restore and DR regularly.
- Use multi-region and multi-AZ architectures.
- Monitor backup and replication health.
- Include dependencies in DR planning.
- Document runbooks and keep them updated.
- Simulate real failure scenarios.
- Continuously improve based on test results and incidents.

19 Production Incident Architecture

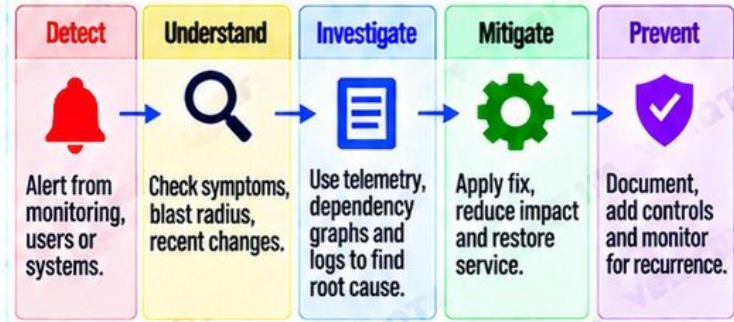
Reconstructing failure through blast radius, dependency graphs, telemetry, recent changes and mitigation

1 WHAT IS PRODUCTION INCIDENT ARCHITECTURE?

- A structured way to understand what failed, how far it spread and how to recover.
- Uses telemetry, logs, metrics, traces, events and recent changes to find the root cause.
- Helps reduce downtime and prevent repeat issues.
- Includes blast radius analysis, dependency mapping and clear mitigation steps.



2 INCIDENT INVESTIGATION FLOW



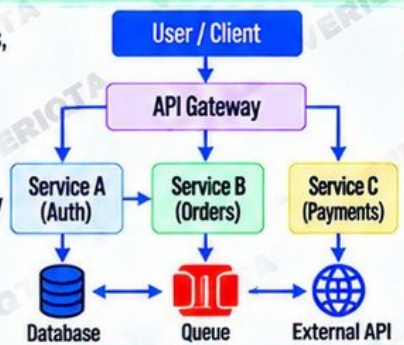
3 BLAST RADIUS ANALYSIS

- Identifies how far the failure has spread.
- Looks at affected services, users, data and regions.
- Helps prioritize the right fix.
- Reduces the risk of missing hidden impact.



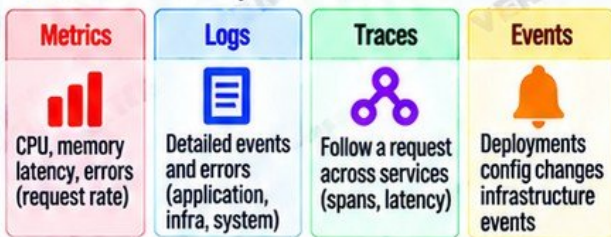
4 DEPENDENCY GRAPH

- Shows how services, databases, queues and external systems are connected.
- Helps find where the failure started and what it can affect.
- Includes internal and third-party dependencies.
- Useful for impact analysis and faster troubleshooting.



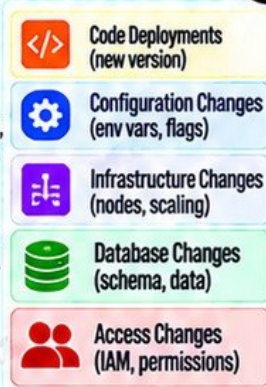
5 TELEMETRY FOR INCIDENTS

- Use metrics to see performance and errors.
- Use logs to find detailed error messages.
- Use traces to follow requests across services.
- Use events to check infrastructure changes.
- Combine all telemetry to find the root cause.



6 RECENT CHANGES ANALYSIS

- Check what changed before the incident.
- Look at deployments, configuration changes, infrastructure updates and feature flags.
- Identify risky changes that can cause the issue.
- Correlate the timing with the incident start.



7 MITIGATION STRATEGIES

- Contain the issue and reduce the blast radius.
- Roll back recent changes if needed.
- Scale resources if it is a capacity issue.
- Fail over to a healthy region or standby system.
- Disable or isolate the failing component.
- Communicate with stakeholders regularly.

8 INCIDENT ARCHITECTURE LIFECYCLE



9 BEST PRACTICES

- Have clear runbooks.
- Maintain a dependency map.
- Use centralized logging and monitoring.
- Correlate metrics, logs, traces and events.
- Track and review recent changes.
- Set up alerting with meaningful thresholds.
- Test incident response and failover regularly.
- Document incidents and implement improvements.

10 COMMON TOOLS



COMMON MISTAKES

- Ignoring early warning signs.
- Not checking recent changes.
- Looking at only one type of telemetry.
- Missing external dependencies.
- No clear incident ownership.
- Poor communication during the incident.
- No post-incident review.

20 Design a Production Platform

End-to-end architecture exercise combining traffic, compute, data, security, observability, deployment and recovery

1 PROJECT OVERVIEW

- Design a complete, real-world production platform from scratch.
- Combine all key areas: traffic, compute, data, security, observability, deployment and recovery.
- Focus on scalability, reliability, security and operability.
- Apply best practices and real production thinking.
- Document the architecture, components and decisions.

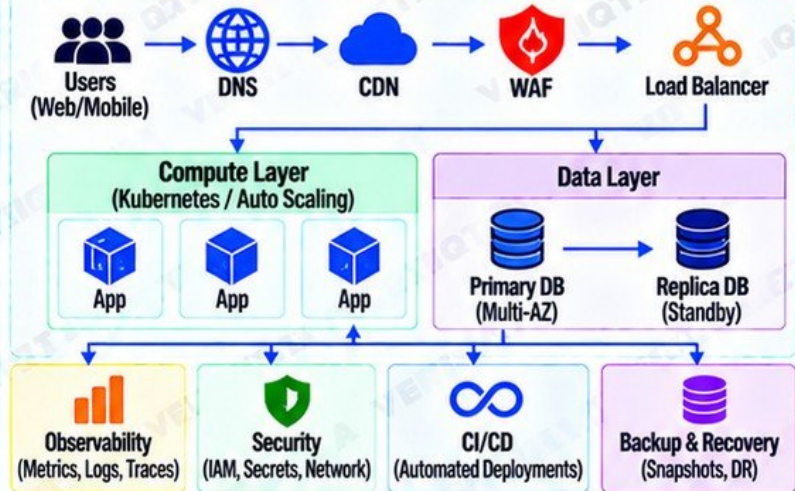
BUSINESS SCENARIO

A global web application for millions of users with high availability, secure access, automated deployments and full observability.

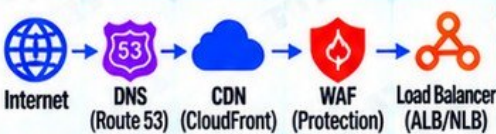
REQUIREMENTS

- Highly available and scalable
- Secure and compliant
- Automated deployment pipeline
- Real-time monitoring and alerting
- Backup and disaster recovery
- Cost-effective and maintainable

2 HIGH-LEVEL ARCHITECTURE



3 TRAFFIC AND NETWORK LAYER



- Handle global traffic with DNS and CDN.
- Protect with WAF and DDoS mitigation.
- Distribute traffic across multiple regions and zones.
- Use health checks and automatic failover.

4 COMPUTE AND APPLICATION LAYER



- Run applications in containers.
- Use Kubernetes or managed services.
- Enable auto scaling based on demand.
- Design for stateless applications.
- Use multiple availability zones.

5 DATA LAYER



- Use managed databases (e.g. RDS, Aurora).
- Enable read replicas for performance.
- Store files in object storage (e.g. S3).
- Implement encryption and access controls.
- Plan for backups and point-in-time recovery.

6 SECURITY LAYER



- Use least privilege IAM roles and policies.
- Store secrets securely (e.g. AWS Secrets Manager).
- Use network segmentation and security groups.
- Encrypt data in transit and at rest.
- Implement threat detection and audit logging.

7 OBSERVABILITY LAYER



- Collect metrics, logs and traces from all components.
- Set up centralized observability stack.
- Create dashboards for key services.
- Set up alerting for critical issues.
- Monitor application, infrastructure and business metrics.

8 CI/CD AND DEPLOYMENT LAYER



- Use version control (e.g. GitHub).
- Automate build, test and security scans.
- Build container images and store in registry.
- Deploy using CI/CD (e.g. GitHub Actions, Jenkins).
- Use blue-green or canary deployments.
- Enable automatic rollback.

9 BACKUP AND DISASTER RECOVERY



- Automate backups for all critical data.
- Replicate data to another region.
- Define RTO and RPO targets.
- Test restore process regularly.
- Implement multi-region failover.

10 END-TO-END FLOW



KEY TAKEAWAYS

- Design for scalability, reliability and security.
- Integrate all components into a complete platform.
- Use automation for deployment, monitoring and recovery.
- Plan for failures and test recovery regularly.
- Apply real-world best practices and document everything.